



# Admin Panel

## Kılavuzu

Pratik Tarifler · Operasyon Merkezi

Next.js 14

TypeScript

Firebase Admin

Tailwind CSS

Sürüm 1.0.0 · Haziran 2026

Bu kılavuz, Pratik Tarifler admin panelinin  
tüm özelliklerini, mimarisini ve kullanımını anlatır.



# İçindekiler

|                                   |    |
|-----------------------------------|----|
| 1. Giriş — Admin Panel Nedir?     | 3  |
| 2. Mimari Genel Bakış             | 4  |
| 3. Teknoloji Yığını               | 5  |
| 4. Sayfalar ve Özellikler         | 6  |
| 4.1 Dashboard — Genel Bakış       | 6  |
| 4.2 Tarif Yönetimi                | 7  |
| 4.3 Kullanıcı Yönetimi            | 8  |
| 4.4 Push Bildirim Yönetimi        | 9  |
| 4.5 Görsel Pipeline Kontrolü      | 10 |
| 4.6 Duyuru ve İpucu Yönetimi      | 11 |
| 4.7 Analytics Dashboard           | 12 |
| 4.8 Destek Talep Yönetimi         | 13 |
| 4.9 Remote Config & A/B Test      | 14 |
| 4.10 Sistem Ayarları & Audit Log  | 15 |
| 5. API Rotaları Referansı         | 16 |
| 6. Yetkilendirme ve Roller (RBAC) | 18 |
| 7. Kurulum ve Deploy              | 19 |
| 8. Geliştirici Rehberi            | 20 |

# 1. Giriş — Admin Panel Nedir?

**Pratik Tarifler Admin Panel**, mobil uygulamanın tüm operasyonel ihtiyaçlarını tek bir yerden yönetmeni sağlayan web tabanlı bir kontrol merkezidir. Tek başına bir founder iken de, 20 kişilik bir ekip olduğunda da çalışacak şekilde tasarlandı.

## 1.1 Hangi sorunları çözer?

- **İçerik yönetimi:** 2500+ tarifi UI üzerinden ekle/düzenle/sil. JSON dosyalarına dokunma.
- **Pazarlama:** Push bildirim kampanyaları oluştur, segment ile gönder, performansını ölç.
- **Müşteri desteği:** Destek taleplerini cevapla, Premium grant et, hesap yönet.
- **Analitik:** DAU, MRR, dönüşüm hunisi gibi kritik metrikleri tek panelde gör.
- **Operasyonel hız:** Server kod dağıtmadan feature flag'leri değiştir (Remote Config).
- **Görsel üretim:** 2500 tarif için görsel pipeline'ı izle ve kontrol et.



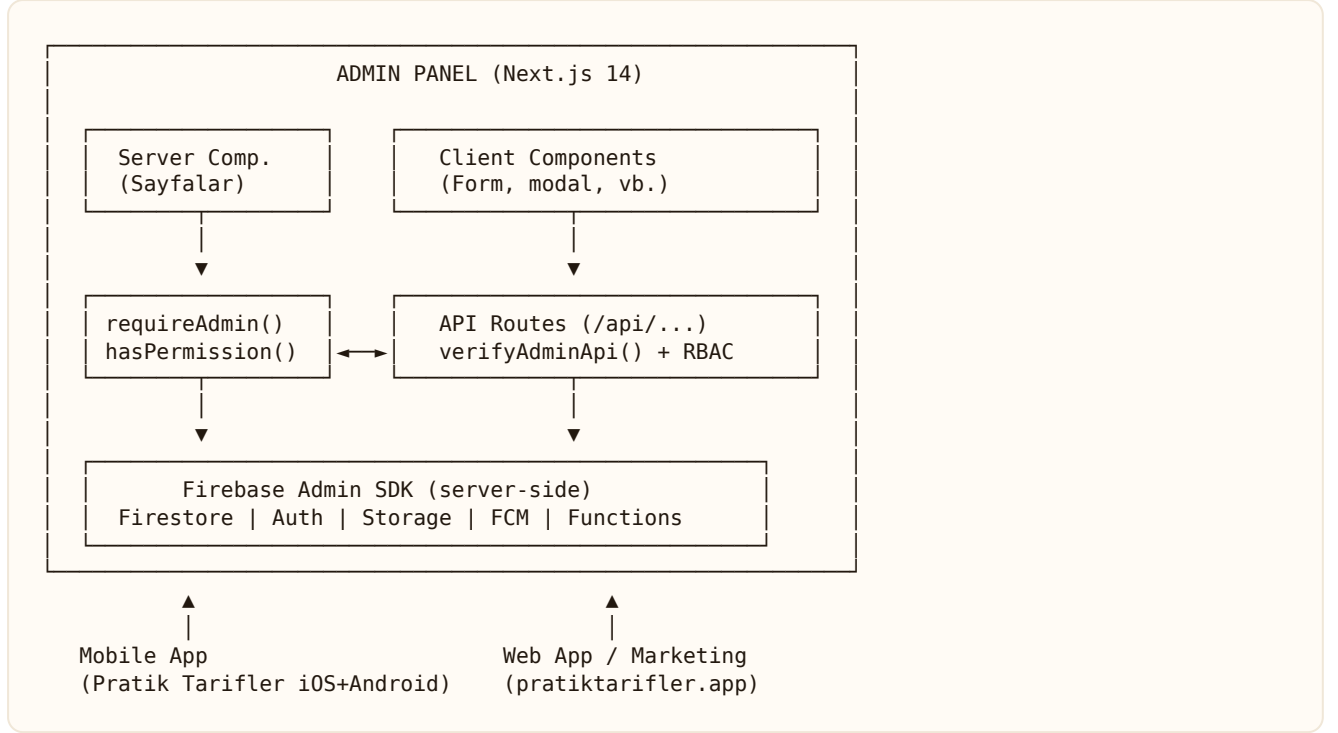
**Felsefe:** Mobil uygulama "ne yapar" → admin panel "kim ne yapsın". İkisi de Firebase backend'i üzerinde çalışır, dolayısıyla **tek doğru kaynak vardır**.

## 1.2 Ne kazandırır?

| Önce                                                        | Sonra                                    |
|-------------------------------------------------------------|------------------------------------------|
| JSON dosyasını manuel düzenle → seed et → deploy et (15 dk) | Form doldur → Kaydet (30 sn)             |
| Push için Firebase Console + manuel kullanıcı listesi       | Segment seç → Önzileme → Gönder (1 dk)   |
| Premium grant için Firestore'u açıp manuel düzenle          | Kullanıcıya tıkla → "Premium ver" butonu |
| Analytics için 5 farklı dashboard aç                        | Tek sayfada 24+ KPI                      |

## 2. Mimari Genel Bakış

### 2.1 Üst seviye akış



### 2.2 Veri akışı örneği — Push gönderim

1. Admin `/notifications/new` sayfasında form doldurur
2. "Gönder" tıkla → `POST /api/notifications/send`
3. API: `verifyAdminApi()` → cookie doğrula
4. API: `hasPermission(user, "write")` → rol kontrolü
5. Firestore query: segment kriterine uyan kullanıcıların FCM tokenlarını çek
6. FCM Admin SDK: 500'lük gruplarda `sendEachForMulticast()`
7. `push_campaigns` koleksiyonuna kayıt + `audit_logs`
8. Response: `{ campaign_id, delivered, failed }`
9. UI: success toast + listede yeni kampanya

### 2.3 Güvenlik katmanları

| Katman           | Görev                                                     |
|------------------|-----------------------------------------------------------|
| Middleware       | Cookie yoksa /login redirect                              |
| Server Component | <code>requireAdmin()</code> → ADMIN_EMAILS kontrolü       |
| API Route        | <code>verifyAdminApi()</code> → JSON 401 dön              |
| İş mantığı       | <code>hasPermission(user, action)</code> → RBAC matrisi   |
| Audit log        | Tüm yazma işlemleri <code>audit_logs</code> koleksiyonuna |

## 3. Teknoloji Yığını

### 3.1 Frontend

| Teknoloji         | Sürüm  | Görev                                            |
|-------------------|--------|--------------------------------------------------|
| Next.js           | 14.2.5 | React framework, App Router, Server Components   |
| React             | 18.3   | UI library                                       |
| TypeScript        | 5.5    | Tip güvenliği (strict mode)                      |
| Tailwind CSS      | 3.4    | Utility-first CSS                                |
| shadcn/ui pattern | —      | Copy-paste UI bileşenleri (Radix tabanlı)        |
| Lucide Icons      | 0.424  | İkon kütüphanesi (2000+ ikon)                    |
| TanStack Query    | 5.51   | Server state yönetimi                            |
| TanStack Table    | 8.20   | Veri tabloları (filtreleme, sıralama, sayfalama) |
| Recharts          | 2.12   | Grafik ve görselleştirme                         |
| React Hook Form   | 7.52   | Form yönetimi                                    |
| Zod               | 3.23   | Şema validasyonu                                 |
| date-fns          | 3.6    | Tarih işlemleri (Türkçe locale)                  |

### 3.2 Backend

| Teknoloji                | Sürüm | Görev                                       |
|--------------------------|-------|---------------------------------------------|
| Firebase Admin SDK       | 12.4  | Server-side Firebase erişimi                |
| Firestore                | —     | Ana veritabanı (recipes, users, audit_logs) |
| Firebase Auth            | —     | Admin girişi (Session Cookie)               |
| Firebase Cloud Messaging | —     | Push gönderimi                              |
| Firebase Storage         | —     | Görsel barındırma                           |
| Firebase Remote Config   | —     | Feature flags, A/B test                     |

### 3.3 Deploy ortamı

| Servis           | Notlar                                 |
|------------------|----------------------------------------|
| Vercel           | Önerilen — Next.js native              |
| Firebase Hosting | Alternatif (Next.js Cloud Run gerekir) |
| Self-hosted      | Docker + nginx + Node.js 20            |

### 3.4 Geliştirme araçları

| Araç                   | Kullanım                   |
|------------------------|----------------------------|
| ESLint + Prettier      | Kod kalitesi ve formatlama |
| Jest + Testing Library | Birim testler              |
| Playwright             | E2E testleri               |
| GitHub Actions         | CI/CD pipeline             |

## 4. Sayfalar ve Özellikler

### 4.1 Dashboard — Genel Bakış

Yol: /dashboard

Admin paneline giriş yaptıktan sonra gördüğün ana sayfa. Sistemin kritik metrikleri tek bakışta.

#### Özellikler

##### 8 KPI Kartı

- **Toplam Kullanıcı:** 24.580 + son 7 gün artış
- **Günlük Aktif:** DAU + MAU karşılaştırma
- **Aylık Gelir (MRR):** 38.500 ₺ + aktif abone sayısı
- **Toplam Tarif:** 2.500 + görselli sayısı
- **Ortalama Puan:** 4.6 ★
- **Crash-Free Rate:** %99.7
- **Deneme Aktif:** 187 trial kullanıcı
- **Aksiyon Gereken:** Açık destek + onay bekleyen görsel

##### Son 30 Gün Trend Grafiği

Günlük indirme ve aktif kullanıcı eğilimini gösterir. Recharts ile interactive.

##### Son Aktiviteler Akışı

Audit log'tan gerçek zamanlı son 10 işlem. "Yeni kullanıcı kaydı", "Premium satın alma", "Yeni tarif eklendi" vb.

##### Hızlı İşlem Kartları

4 popüler işleme tek tıkla erişim: Yeni Tarif Ekle, Push Bildirim Gönder, Görsel Pipeline, Destek Talepleri.

### 4.2 Tarif Yönetimi

Yol: /recipes

## Liste sayfası

- **Filtreler:** Mutfak, zorluk, görsel durumu (hazır/bekliyor/hatalı), tier (free/premium)
- **Arama:** Tarif adı veya ID
- **Sayfalama:** 50'şer kayıt
- **Bulk işlemler:** JSON içe aktar, dışa aktar, çoklu seçimle sil/yayından kaldır
- **Tablo sütunları:** Tarif | Mutfak | Zorluk | Süre | Tier | Görsel | Puan | Durum | İşlemler

## Yeni tarif / Düzenleme formu

Tam tarif yapısını UI üzerinden girebileceğin form:

- **Temel bilgiler:** Başlık, açıklama, mutfak, zorluk, dil, süre, porsiyon, öğün tipi, diyet etiketleri
- **Malzemeler:** Dinamik liste — miktar + token + not
- **Adımlar:** Sıralı, her adım için başlık + gövde + opsiyonel sayaç (saniye)
- **Püf noktaları:** Opsiyonel madde listesi
- **Görsel:** Manuel yükle veya AI ile otomatik üret
- **Yayın ayarları:** Premium işareti, editöryal seçim, yayında/taslak

✓ **Çeviri otomasyonu:** Türkçe kayıt sonrası 12 dile çeviri tetiklenir (OpenAI Batch API ile).

## 4.3 Kullanıcı Yönetimi

Yol: `/users`

### Liste ve filtreler

- 3 üst KPI: Toplam, Aktif (30 gün), Premium aboneler
- Filtreler: Abonelik durumu (free/trial/active/cancelled/expired), dil, ban durumu
- Arama: E-posta, kullanıcı adı, UID

### Tabloda gösterilen bilgiler

| Sütun     | İçerik                                      |
|-----------|---------------------------------------------|
| Kullanıcı | Avatar + isim + e-posta                     |
| Abonelik  | Durum rozeti + plan (Aylık/Yıllık)          |
| Dil       | TR, EN, vb.                                 |
| Kayıt     | Kayıt tarihi                                |
| Son Aktif | "2 saat önce" formatında                    |
| İşlemler  | Premium ver, E-posta gönder, Yasakla, Detay |

### İşlemler (API: PATCH `/api/users`)

- grant\_premium:** Kullanıcıya manuel Premium (varsayılan 365 gün, özelleştirilebilir)
- revoke\_premium:** Premium'u geri al
- ban:** Hesabı yasakla, sebep belirt
- unban:** Yasaklamayı kaldır
- delete:** KVKK uyumlu tam silme (Firestore + Storage + Auth)

## 4.4 Push Bildirim Yönetimi

Yol: `/notifications` (liste) ve `/notifications/new` (yeni)

### Kampanya oluşturma adımları

- İçerik:** Türkçe başlık + mesaj (zorunlu), 12 dile çeviri (opsiyonel veya otomatik)
- Görsel:** Bildirim büyük görseli (opsiyonel, URL)
- Deep Link:** `pratiktarifler://recipe/xyz` formatında uygulama içi yönlendirme
- Hedef Kitle (Segment):**

- Tüm kullanıcılar (~24.580)
- Sadece Premium aboneler (~942)
- Sadece ücretsiz kullanıcılar (~23.638)
- Dile göre (TR, EN, vb.)
- Ülkeye göre (Türkiye, Almanya, vb.)
- Özel Firestore sorgusu

5. **Önizleme:** iOS/Android cihaz mockup ile gerçek görüntü

6. **Test Gönder:** Sadece kendi cihazına

7. **Zamanla veya Şimdi Gönder**

## Listede performans metrikleri

| Metrik       | Anlamı                                              |
|--------------|-----------------------------------------------------|
| Alıcı        | Push token'ı olan hedef kullanıcı sayısı            |
| İletilen     | FCM'in başarılı kabul ettiği sayı (%91 ortalama)    |
| Açılan       | Kullanıcının bildirime tıkladığı sayı               |
| Açılma oranı | $Açılan / İletilen \times 100$ (sektör ort. %18-25) |

## 4.5 Görsel Pipeline Kontrolü

Yol: `/images`

2500 tarif için görsel üretim süreci tek panelden yönetilir. Pipeline 5 kaynaktan (Unsplash, Pexels, Pixabay, DALL·E 3, Flux Pro) hibrit şekilde görsel toplar.

### Ana metrikler

|                          |                       |
|--------------------------|-----------------------|
| İLERLEME<br><b>%75.0</b> | HAZIR<br><b>1.875</b> |
| BEKLEYEN<br><b>597</b>   | HATALI<br><b>28</b>   |

### Pipeline yapılandırması

- Paralellik:** Eşzamanlı işlem sayısı (varsayılan: 5)
- AI çağrı limiti:** Run başına maks DALL·E 3 / Flux çağrısı (varsayılan: 100)
- Kaynak öncelik sırası:** Sürükle-bırak ile Unsplash → Pexels → Pixabay → DALL·E → Flux
- Maliyet tahmini:** Kalan tarifler için tahmini USD (\$38.50 örneğin)

### İş kuyruğu

Anlık olarak çalışan ve bekleyen iş'leri gösterir:

- `queued` — kuyrukta bekliyor
- `searching` — Unsplash/Pexels/Pixabay'da aranıyor
- `generating` — DALL·E veya Flux ile üretiliyor
- `ready` — başarılı, kullanıma hazır
- `failed` — tüm denemeler başarısız (manuel müdahale gerekir)

### İnceleme kuyruğu

AI ile üretilen görseller insan onayı bekler (yiyecek fotoğrafçılığı kalite kontrolü için). Her görsel için:

- ✓ **Onayla** — Storage'a yükle, tarife ata, yayına al
- 🔄 **Tekrar Üret** — Farklı prompt ile yeni deneme
- ✗ **Reddet** — Manuel görsel yükleme talebi olarak işaretle

## 4.6 Duyuru ve İpucu Yönetimi

Yol: /announcements

Uygulama içinde gösterilen banner, modal ve ipuçlarını yönetirsin.

#### 4 içerik türü

| Tür          | Kullanım                        | Örnek                              |
|--------------|---------------------------------|------------------------------------|
| tip          | Kullanıcıyı bilgilendiren ipucu | "💡 Pişirme modu nasıl kullanılır?" |
| announcement | Genel duyuru                    | "🌸 İlkbahar tarifleri eklendi!"    |
| promo        | İndirim/kampanya                | "★ İlk yıllık %50 indirim!"        |
| update       | Sürüm güncelleme bildirimi      | "📱 Sürüm 1.1 — Karanlık mod"       |

#### Gösterim yerleri

- home\_banner — Ana sayfa üst banner
- explore\_card — Keşfet sayfasındaki özel kart
- modal — Tam ekran modal (zorla göster)

#### Hedef kitle ve zamanlama

- Audience: all / premium / free
- Tarih aralığı: Başlangıç + (opsiyonel) bitiş
- Öncelik: Aynı anda birden fazla varsa hangisi önce
- Görüntüleme/tıklama metrikleri otomatik kaydedilir

## 4.7 Analytics Dashboard

Yol: /analytics

Detaylı performans metrikleri 3 bölüm halinde:

### Bölüm 1: Acquisition (Edinme)

| KPI                               | Açıklama                      |
|-----------------------------------|-------------------------------|
| Yeni kullanıcı (bugün/7gün/30gün) | Yeni hesap oluşturma sayıları |
| App Store/Play Store impression   | Mağaza sayfa görüntülemesi    |
| Conversion rate                   | Impression → indirme dönüşümü |

### Bölüm 2: Engagement (Etkileşim)

| KPI                       | Anlamı                                    |
|---------------------------|-------------------------------------------|
| DAU / WAU / MAU           | Günlük / Haftalık / Aylık aktif kullanıcı |
| Stickiness (DAU/MAU)      | Yapışkanlık oranı (hedef: %20+)           |
| D7 retention              | 7. gün hâlâ aktif olan kullanıcı %'si     |
| D30 retention             | 30. gün hâlâ aktif olan                   |
| Mod 1 kullanım (haftalık) | AI buzdolabı tarama sayısı                |

### Bölüm 3: Monetization (Gelir)

| KPI          | Anlamı                                      |
|--------------|---------------------------------------------|
| MRR / ARR    | Aylık / Yıllık tekrarlayan gelir            |
| Toplam abone | Aktif Premium kullanıcı                     |
| Trial → Paid | Deneme süresi sonrası dönüşüm (hedef: %30+) |
| Aylık churn  | İptal eden % (hedef: <%5)                   |
| ARPU         | Kullanıcı başına aylık gelir                |
| LTV          | Yaşam boyu değer                            |

### Dönüşüm Hunisi (Funnel)

8 adımlı tam kullanıcı yolculuğu:

- İndirme (100%) → 24.580
- İlk açılış (93.5%) → 22.980
- Onboarding tamamla (77.0%) → 18.934
- Hesap oluştur (57.8%) → 14.210
- İlk tarif görüntüle (52.4%) → 12.880
- Premium ekranı gör (25.4%) → 6.240
- Deneme başlat (7.6%) → 1.872
- Trial → Paid (3.8%) → 942

## 4.8 Destek Talep Yönetimi

Yol: /support

### Üst KPI'lar

- Açık:** Henüz cevap verilmemiş talepler
- İşlemde:** Yanıt verildi, kullanıcı bekleniyor
- Bugün Çözüldü:** Aktif çözüm performansı

### Filtreler

| Filtre   | Seçenekler                                                  |
|----------|-------------------------------------------------------------|
| Durum    | open · in_progress · waiting_user · resolved · closed       |
| Kategori | bug · feature_request · billing · account · content · other |
| Öncelik  | urgent · high · medium · low                                |
| Atanmış  | Bana atananlar / atanmamış / belirli admin                  |

### Ticket detay görünümü

- Kullanıcı bilgileri (e-posta, abonelik durumu, son aktif)
- Cihaz bilgisi (iOS/Android, app sürümü)
- Mesaj geçmişi (kullanıcı + admin yanıtları)
- Ek dosyalar (ekran görüntüleri, loglar)
- Hızlı aksiyonlar: Premium ver, ban, hesap sil

## 4.9 Remote Config & A/B Test

Yol: `/remote-config`

Uygulama davranışını **kod değişmeden, dağıtım yapmadan** değiştirebilirsin. Firebase Remote Config üzerinde çalışır.

### Örnek anahtarlar

| Anahtar                                | Tip     | Değer             | Etki                          |
|----------------------------------------|---------|-------------------|-------------------------------|
| <code>max_free_scans_per_day</code>    | number  | 3                 | Ücretsiz günlük tarama limiti |
| <code>premium_monthly_price_try</code> | number  | 49                | Aylık Premium fiyatı          |
| <code>feature_voice_commands</code>    | boolean | true              | Sesli komut aktif mi?         |
| <code>feature_dark_mode</code>         | boolean | true              | Karanlık mod                  |
| <code>ai_model</code>                  | string  | claude-sonnet-4-6 | Backend AI modeli             |
| <code>min_app_version</code>           | string  | 1.0.0             | Zorunlu güncelleme alt sınırı |

### Variant koşulları

Her anahtar için birden fazla "variant" tanımlanabilir:

- language:** Sadece TR / EN / DE için farklı değer
- country:** Sadece Türkiye / Almanya için
- app\_version\_min:** 1.1.0 ve üzeri için
- user\_segment:** all / premium / free
- percentage:** Kullanıcıların %25'i için (A/B test)

### A/B Test örneği

#### Onboarding Varyantı (B vs A)

3 ekran vs 5 ekran karşılaştırma

 %50 / %50 ·  8.421 kullanıcı ·  7 gün

**Varyant A:** Tamamlama %72

**Varyant B:** Tamamlama %77.2 (+5.2% ↑) 

## 4.10 Sistem Ayarları & Audit Log

Yol: `/settings`

## Admin kullanıcılar (RBAC)

4 rol seviyesi:

| Rol            | Yetki                                 |
|----------------|---------------------------------------|
| super_admin    | Her şey (admin ekle/kaldır dahil)     |
| content_editor | Tarif, duyuru, push gönderim          |
| support_agent  | Sadece destek + kullanıcı görüntüleme |
| viewer         | Sadece okuma (analytics dashboard)    |

## API anahtarları

3. parti entegrasyonların durumu:

- OpenAI · RevenueCat · Google Vision · Sentry DSN · Stripe
- Anahtarlar Firebase Secrets üzerinde saklanır, UI sadece maskelenmiş gösterir

## Audit Log

Her yazma işlemi log'lanır:

```
{
  admin_uid: "abc123",
  admin_email: "founder@pratiktarifler.app",
  action: "recipe.create",
  details: { recipe_id: "tr-mantar-corbasi", title: "Mantar Çorbası" },
  timestamp: 2026-06-22T08:30:00Z,
  ip: "85.105.x.x"
}
```

12 farklı action tipi: recipe.create/update/delete/bulk\_import,  
user.update/delete/grant\_premium/revoke\_premium, notification.send/schedule/cancel,  
announcement.create/update/delete, config.update, image.regenerate, support.reply/close

## 5. API Rotaları Referansı

Tüm API rotaları Next.js Route Handlers olarak `app/api/...` altında. Her biri önce `verifyAdminApi()` ile cookie doğrular, sonra `hasPermission()` ile RBAC kontrolü yapar.

### Authentication

#### POST /api/auth/session

Firebase ID Token'ı session cookie'ye dönüştürür.

```
// Request
POST /api/auth/session
{ "idToken": "eyJ..." }

// Response 200
{ "ok": true, "email": "admin@..." }

// Response 403
{ "error": "Bu hesap admin paneline erişim yetkisine sahip değil" }
```

#### POST /api/auth/logout

Session cookie'yi siler, /login'e redirect eder.

### Recipes

#### GET /api/recipes?lang=tr&cuisine=turkish&limit=50

```
// Response
{
  "recipes": [
    { "id": "tr-mercimek-corbasi", "title": "Mercimek Çorbası", ... }
  ],
  "count": 47
}
```

#### POST /api/recipes

Yeni tarif oluşturur. `write` yetkisi gerekir. Audit log: `recipe.create`

#### PATCH /api/recipes/[id]

Var olan tarifi günceller. Audit log: `recipe.update`

#### DELETE /api/recipes/[id]

Tarifi siler. `delete` yetkisi gerekir. Audit log: `recipe.delete`

### Users

## PATCH /api/users

```
// Request – Premium ver
{
  "uid": "user_abc",
  "action": "grant_premium",
  "duration_days": 365
}

// Response
{ "ok": true, "expires": 1750000000000 }
```

Desteklenen action'lar: `grant_premium`, `revoke_premium`, `ban`, `unban`

## Notifications

### POST /api/notifications/send

```
// Request
{
  "title": "👁 Bugün ne pişirsem?",
  "body": "Buzdolabını çek, AI sana hızlı bir akşam yemeği önersin!",
  "image_url": "https://...",
  "deep_link": "pratiktarifler://recipe/tr-mercimek-corbasi",
  "segment": {
    "type": "all" // veya "premium", "free", "language", "country", "custom"
  }
}

// Response
{
  "campaign_id": "abc123",
  "delivered": 22341,
  "failed": 239,
  "total": 22580
}
```

Akış:

1. Segment kriterine uyan kullanıcıların FCM token'larını Firestore'dan çek
2. 500'lük batch'ler halinde `adminMessaging.sendEachForMulticast()`
3. `push_campaigns` koleksiyonuna kayıt
4. `audit_logs` 'a yaz

## Images

### POST /api/images/regenerate

```
// Request
{
  "recipe_id": "tr-baklava",
  "force_ai": true // false ise önce Unsplash/Pexels dener
}

// Response
{ "ok": true, "message": "Görsel kuyruğa alındı" }
```

`image_jobs` koleksiyonuna yeni iş ekler. Cloud Function bunu pick edip pipeline'ı tetikler.

## Tüm API rotalarının özet tablosu

| Endpoint                | Method            | Yetki  | Audit Log         |
|-------------------------|-------------------|--------|-------------------|
| /api/auth/session       | POST              | —      | —                 |
| /api/auth/logout        | POST              | —      | —                 |
| /api/recipes            | GET               | read   | —                 |
| /api/recipes            | POST              | write  | recipe.create     |
| /api/recipes/[id]       | PATCH             | write  | recipe.update     |
| /api/recipes/[id]       | DELETE            | delete | recipe.delete     |
| /api/users              | PATCH             | write  | user.*            |
| /api/notifications/send | POST              | write  | notification.send |
| /api/images/regenerate  | POST              | write  | image.regenerate  |
| /api/announcements      | POST/PATCH/DELETE | write  | announcement.*    |
| /api/config             | PATCH             | super  | config.update     |
| /api/support/[id]/reply | POST              | write  | support.reply     |

## 6. Yetkilendirme ve Roller (RBAC)

### 6.1 Rol matrisi

| Aksiyon                | super_admin | content_editor | support_agent | viewer |
|------------------------|-------------|----------------|---------------|--------|
| Tarif oluřtur/düzenle  | ✓           | ✓              | —             | —      |
| Tarif sil              | ✓           | —              | —             | —      |
| Push gönder            | ✓           | ✓              | —             | —      |
| Duyuru oluřtur         | ✓           | ✓              | —             | —      |
| Kullanıcı görüntüle    | ✓           | ✓              | ✓             | ✓      |
| Premium ver/al         | ✓           | ✓              | ✓             | —      |
| Kullanıcı banla        | ✓           | —              | ✓             | —      |
| Kullanıcı sil (KVKK)   | ✓           | —              | —             | —      |
| Destek talebi yanıtla  | ✓           | —              | ✓             | —      |
| Remote Config deęiřtir | ✓           | —              | —             | —      |
| Admin ekle/sil         | ✓           | —              | —             | —      |
| Analytics görüntüle    | ✓           | ✓              | ✓             | ✓      |
| Audit log görüntüle    | ✓           | —              | —             | —      |

### 6.2 Kod implementasyonu

```
// lib/auth.ts

export function hasPermission(user: AdminUser, action: "read" | "write" | "delete" | "super") {
  const matrix = {
    super_admin: ["read", "write", "delete", "super"],
    content_editor: ["read", "write"],
    support_agent: ["read"],
    viewer: ["read"],
  };
  return matrix[user.role].includes(action);
}

// Kullanım – API route içinde
const user = await verifyAdminApi();
if (!user) return NextResponse.json({ error: "Yetkisiz" }, { status: 401 });
if (!hasPermission(user, "delete")) {
  return NextResponse.json({ error: "Yetkin yetersiz" }, { status: 403 });
}
```

### 6.3 Admin nasıl atanır?

1. `.env` dosyasındaki `ADMIN_EMAILS` listesine e-posta ekle
2. Firebase Auth Custom Claim ile rol ata:

```
// Firebase Console veya CLI
firebase auth:set-custom-claims user@example.com '{"role":"content_editor"}'
```

3. Vercel'de environment variable güncelle, redeploy

## 7. Kurulum ve Deploy

### 7.1 Yerel geliştirme

```
# 1. Bağımlılıkları kur
cd admin_panel
npm install

# 2. .env hazırla
cp .env.example .env
# Firebase Console'dan Service Account JSON indir, değerleri .env'ye yapıştır

# 3. Geliştirme sunucusu
npm run dev
# http://localhost:3000
```

### 7.2 Üretim build

```
npm run build
npm start
# http://localhost:3000 (production mode)
```

### 7.3 Vercel deploy (önerilen)

```
# Vercel CLI kur
npm install -g vercel

# Deploy
cd admin_panel
vercel --prod

# Environment variables Vercel dashboard'dan eklenir:
# Settings → Environment Variables → tüm .env değerlerini ekle
```

### 7.4 Custom domain

Vercel dashboard → Domains → `admin.pratiktarifler.app` ekle. CNAME ayarı domain registrar'dan yapılır.

### 7.5 Firebase Service Account oluşturma

1. Firebase Console → Project Settings → Service Accounts
2. "Generate new private key" → JSON dosyası indir
3. JSON'daki `project_id`, `client_email`, `private_key` değerlerini `.env` 'ye yapıştır
4. **ÖNEMLİ:** JSON dosyasını asla Git'e commit etme!

## 7.6 İlk admin oluşturma

```
# scripts/seed_admin.ts dosyasını çalıştır
npx tsx scripts/seed_admin.ts \
  --email founder@pratiktarifler.app \
  --role super_admin

# veya manuel:
# 1. Firebase Authentication'da kullanıcı oluştur
# 2. ADMIN_EMAILS listesine ekle (.env)
# 3. Custom claim ata
```

## 7.7 Güvenlik kontrol listesi

- ☐ `.env` dosyası Git'te değil mi?
- ☐ `FIREBASE_PRIVATE_KEY` Vercel secret olarak mı?
- ☐ `ADMIN_EMAILS` sadece güvenirli e-postaları mı içeriyor?
- ☐ Firestore rules: admin\_users koleksiyonu sadece super\_admin tarafından mı yazılabilir?
- ☐ Audit log retention policy ayarlandı mı? (önerilen: 1 yıl)
- ☐ Vercel'de "Password Protection" aktif mi?
- ☐ Custom domain için SSL aktif mi?

## 8. Geliştirici Rehberi

### 8.1 Yeni sayfa eklemek

```
# 1. Klasör oluştur
mkdir -p "app/(dashboard)/yeni-sayfa"

# 2. page.tsx yaz
cat > "app/(dashboard)/yeni-sayfa/page.tsx" << 'EOF'
export default async function YeniSayfa() {
  return <div>İçerik</div>;
}
EOF

# 3. Sidebar'a navigasyon ekle
# components/layout/sidebar.tsx → nav array'ine ekle
```

### 8.2 Yeni API rotası eklemek

```
# 1. Klasör + route.ts
mkdir -p app/api/yeni-endpoint
cat > app/api/yeni-endpoint/route.ts << 'EOF'
import { NextRequest, NextResponse } from "next/server";
import { verifyAdminApi, hasPermission } from "@lib/auth";
import { logAudit } from "@lib/audit";

export async function POST(req: NextRequest) {
  const user = await verifyAdminApi();
  if (!user) return NextResponse.json({ error: "Yetkisiz" }, { status: 401 });
  if (!hasPermission(user, "write")) {
    return NextResponse.json({ error: "Yetkin yetersiz" }, { status: 403 });
  }

  const body = await req.json();
  // İş mantığı...

  await logAudit(user, "custom.action", { details: body });
  return NextResponse.json({ ok: true });
}
EOF
```

### 8.3 Firestore koleksiyon yapısı

```
// Mobil uygulama ile paylaşılan koleksiyonlar:
recipes_tr, recipes_en, recipes_de, ... // Dile göre tarif koleksiyonu
users // Kullanıcı dokümanları
push_tokens // FCM tokenları (kullanıcı altında)

// Sadece admin paneli yazar:
push_campaigns // Gönderilen push kampanyaları
announcements // Duyuru ve ipuçları
image_jobs // Görsel üretim kuyruğu
support_tickets // Destek talepleri
admin_users // Admin kullanıcılar (RBAC)
audit_logs // İşlem kayıtları
remote_config_keys // Custom config anahtarları
```

## 8.4 Test yazma

```
# Birim test örneği – Jest
// __tests__/utils.test.ts
import { formatCurrency } from "@lib/utils";

test("formatCurrency Türk lirası formatı", () => {
  expect(formatCurrency(49000)).toBe("₺49.000");
});

# E2E test örneği – Playwright
// e2e/login.spec.ts
import { test, expect } from "@playwright/test";

test("admin başarılı giriş", async ({ page }) => {
  await page.goto("http://localhost:3000/login");
  await page.fill('input[type="email"]', "admin@pratiktarifler.app");
  await page.fill('input[type="password"]', "test123");
  await page.click('button[type="submit"]');
  await expect(page).toHaveURL(/\/dashboard/);
});

# Çalıştır
npm test # Jest
npm run test:e2e # Playwright
```

## 8.5 CI/CD pipeline

[.github/workflows/deploy.yml](#) :

```
name: Deploy Admin Panel
on:
  push:
    branches: [main]
    paths: ['admin_panel/**']

jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: 20 }
      - run: cd admin_panel && npm ci
      - run: cd admin_panel && npm run typecheck
      - run: cd admin_panel && npm run lint
      - run: cd admin_panel && npm test
      - uses: amondnet/vercel-action@v25
        with:
          vercel-token: ${ secrets.VERCEL_TOKEN }
          vercel-args: '--prod'
          working-directory: ./admin_panel
```